

Segmentación de Imágenes SEM mediante AutoML: Comparación de Estrategias de Segmentación

Autor

Resumen—Se presenta un enfoque de aprendizaje automático para la detección de zonas frágiles y dúctiles en imágenes obtenidas mediante microscopía electrónica de barrido. El estudio compara modelos del estado del arte basados en transformers con estrategias de segmentación propuestas, integradas en un framework AutoML. La selección del modelo se realiza a partir de métricas de desempeño relevantes, considerando explícitamente las limitaciones computacionales del entorno de ejecución. Los resultados muestran la viabilidad del enfoque bajo recursos restringidos y evidencian el impacto del uso de aumentación de datos y transferencia de aprendizaje en la calidad de la segmentación.

I. INTRODUCCIÓN

La caracterización de los mecanismos de fractura en materiales es un problema central en la ciencia e ingeniería de materiales, debido a su impacto directo en la evaluación del desempeño mecánico y la confiabilidad estructural. En particular, la identificación de zonas asociadas a comportamientos frágiles y dúctiles a partir de microestructuras proporciona información clave para el análisis del fallo y el diseño de materiales.

Las imágenes obtenidas mediante microscopía electrónica de barrido (SEM) constituyen una de las principales herramientas para el estudio de superficies de fractura y microestructuras. No obstante, el análisis tradicional de este tipo de imágenes se basa en inspección visual experta, lo cual introduce subjetividad, limita la reproducibilidad de los resultados y dificulta su aplicación a grandes volúmenes de datos.

En este contexto, las técnicas de aprendizaje automático y, en particular, los modelos de visión por computador han mostrado un potencial significativo para automatizar tareas de clasificación y segmentación en imágenes complejas. Recientemente, modelos basados en transformers, como Vision Transformer (ViT) y Swin Transformer, han alcanzado resultados destacados en diversas tareas de análisis de imágenes, posicionándose como referentes del estado del arte. Sin embargo, su aplicación práctica suele estar condicionada por la disponibilidad de datos anotados y recursos computacionales elevados.

Estas limitaciones motivan la exploración de estrategias alternativas que permitan aprovechar modelos de alto desempeño bajo restricciones de cómputo y datos. En este trabajo se consideran tanto modelos del estado del arte como enfoques propuestos basados en la descomposición espacial de la imagen y el uso de clasificadores preentrenados, con el objetivo de transferir conocimiento desde tareas de clasificación hacia problemas de segmentación.

Adicionalmente, se adopta un enfoque AutoML para automatizar la selección de modelos y el ajuste de hiperparámetros, empleando metaheurísticas y considerando explícitamente las restricciones impuestas por un entorno de cómputo limitado. El objetivo de este estudio es evaluar la viabilidad y el desempeño de estas estrategias para la detección de zonas frágiles y dúctiles en imágenes SEM, comparando modelos del estado del arte con enfoques alternativos bajo condiciones realistas de entrenamiento y evaluación.

II. METODOLOGÍA BASADA EN AUTOML

La resolución del problema de segmentación se abordó mediante un enfoque de AutoML diseñado específicamente para garantizar modularidad, reproducibilidad y comparabilidad sistemática entre distintas configuraciones experimentales. La metodología no se limita a la automatización del entrenamiento, sino que establece una arquitectura formal que define claramente cada etapa del flujo de datos y permite intercambiar componentes de manera controlada.

Arquitectura general del pipeline

El núcleo metodológico se basa en un pipeline lineal de procesamiento, en el cual los datos fluyen secuencialmente a través de nodos bien definidos. Este pipeline sigue el esquema general:

Dataset inicial → Nodo de aumentación → Nodo de modelo
→ Nodo de evaluación → Resultados finales (1)

Cada etapa del pipeline encapsula una responsabilidad única y explícita, evitando dependencias implícitas y facilitando el análisis aislado del impacto de cada decisión metodológica. La coordinación de este flujo está a cargo de una clase orquestadora central, denominada *AutoML*, cuya función es estrictamente organizativa.

El diseño del framework se fundamenta en el uso extensivo de interfaces abstractas. Todas las etapas del pipeline implementan contratos formales definidos a nivel de interfaz, lo que permite desacoplar completamente la lógica de alto nivel de las implementaciones concretas. Este enfoque garantiza que diferentes estrategias —por ejemplo, distintos esquemas de validación o arquitecturas de red— puedan sustituirse sin alterar la estructura global del sistema.

La metodología adopta el principio de composición: la clase *AutoML* no implementa lógica de entrenamiento, aumentación ni evaluación, sino que recibe instancias de nodos que cumplen interfaces predefinidas y las ejecuta en el orden correcto. Esto resulta en un núcleo extremadamente estable y fácil de auditar desde el punto de vista metodológico.

II-A. Nodo de aumentación y partición de datos

El primer componente del pipeline es el nodo de aumentación de datos, responsable de transformar el dataset original y generar las particiones de entrenamiento y validación. Este nodo recibe el dataset completo y produce una lista de pares (D_{train}, D_{val}) lo que permite soportar tanto divisiones simples como esquemas de validación cruzada k-fold.

Desde el punto de vista metodológico, esta decisión es clave: todas las transformaciones de datos y estrategias de partición quedan formalizadas en un único punto del pipeline, evitando fugas de información y asegurando que todos los modelos evaluados operen bajo condiciones comparables.

II-B. Nodo de modelo

El nodo de modelo constituye el núcleo computacional del sistema. Para cada par de datasets generado por el nodo de aumentación, este componente entrena una instancia del modelo especificado y posteriormente genera predicciones sobre el conjunto de validación. El resultado de esta etapa es una colección estructurada de pares de máscaras predichas y reales, organizada por pliegue cuando corresponde.

Cada arquitectura se encapsula en una implementación concreta de este nodo, manteniendo la misma interfaz externa. Esto permite comparar modelos de distinta naturaleza bajo un marco experimental idéntico, aislando el efecto de la arquitectura del resto del pipeline.

II-C. Nodo de evaluación

El nodo de evaluación recibe exclusivamente las predicciones y las máscaras reales producidas por el nodo de modelo. Su responsabilidad es calcular las métricas de desempeño de forma agregada y consistente. Internamente, este nodo delega el cálculo a evaluadores individuales, cada uno asociado a una métrica específica.

Desde el punto de vista metodológico, este desacoplamiento garantiza que las métricas no influyan en el proceso de entrenamiento ni en la generación de predicciones, evitando sesgos y permitiendo incorporar nuevas métricas sin modificar etapas previas del pipeline.

II-D. Orquestación y ejecución automática

La clase *AutoML* actúa como el punto único de ejecución del experimento. Su método principal ejecuta secuencialmente los nodos de aumentación, modelo y evaluación, y retorna un conjunto final de métricas. Este diseño minimalista reduce la complejidad cognitiva del sistema y asegura que el flujo experimental sea explícito y reproducible.

La automatización se logra mediante la ejecución repetida del pipeline con distintas combinaciones de nodos y configuraciones. De este modo, el proceso de selección de modelos e hiperparámetros se formaliza como una exploración sistemática del espacio de configuraciones, donde el criterio de selección se basa exclusivamente en las métricas producidas por el nodo evaluador.

La construcción concreta de los nodos no se realiza directamente en los scripts experimentales, sino mediante funciones

de configuración ubicadas en un módulo dedicado. Este enfoque separa claramente la definición del experimento de la lógica interna del framework, mejorando la legibilidad, reduciendo errores de configuración y facilitando la reproducibilidad de los resultados.

Esta metodología basada en *AutoML* establece una base sólida y extensible para todo el desarrollo posterior, dentro de un marco experimental unificado y controlado.

III. DESCRIPCIÓN DEL DATASET

IV. ESTRATEGIAS DE SEGMENTACIÓN

La segmentación de zonas frágiles y dúctiles se aborda mediante un conjunto de estrategias complementarias que responden a distintos niveles de complejidad y granularidad en la representación de la imagen. En primer lugar, se consideran arquitecturas basadas en Vision Transformers y Swin Transformers, seleccionadas por su capacidad para modelar dependencias espaciales de largo alcance y por su consolidación como enfoques de referencia en tareas de segmentación densa. Estos modelos permiten aprender representaciones globales y jerárquicas directamente a partir de la imagen completa, proporcionando una base sólida para la identificación precisa de regiones con comportamiento mecánico diferenciado.

De forma complementaria, se exploran estrategias de segmentación indirecta que reutilizan conocimiento previamente adquirido en tareas de clasificación. En este enfoque, la información semántica extraída por modelos entrenados para discriminar regiones frágiles y dúctiles se emplea como guía para definir esquemas de segmentación basados en particiones espaciales adaptativas. En particular, se consideran métodos que descomponen la imagen en regiones locales cuya resolución y extensión se ajustan dinámicamente en función de la complejidad visual y la confianza de la clasificación, permitiendo una transición controlada entre análisis global y local.

Este planteamiento unifica modelos de segmentación directa y estrategias guiadas por clasificación bajo un mismo marco conceptual, facilitando la comparación entre enfoques de distinta naturaleza y sentando las bases para la incorporación progresiva de métodos alternativos de particionado espacial en subsecciones posteriores.

IV-A. Vision Transformer (ViT)

La estrategia de segmentación basada en *Vision Transformer* (ViT) se diseñó siguiendo un enfoque modular que separa explícitamente la definición arquitectónica del modelo del proceso de entrenamiento y evaluación. Esta decisión metodológica permite desacoplar los aspectos conceptuales del modelo de los detalles operativos, facilitando su integración en el framework de *AutoML* y asegurando comparabilidad con otras estrategias de segmentación.

IV-A1. Arquitectura encoder-decoder

El modelo adopta una arquitectura de tipo *encoder-decoder*, donde el encoder se basa en un Transformer y el decoder en una red neuronal convolucional. Esta combinación permite explotar la capacidad del mecanismo de autoatención para

capturar dependencias globales, mientras que el decoder convolucional reconstruye la información espacial necesaria para una segmentación densa a nivel de píxel.

El encoder transforma la imagen de entrada en una secuencia de representaciones latentes mediante un proceso de partición en parches no solapados. Cada parche es proyectado a un espacio de alta dimensionalidad, generando una secuencia de *embeddings* que reemplaza la representación basada en píxeles. Dado que los Transformers carecen de noción explícita de orden espacial, se incorporan codificaciones posicionales aprendibles, las cuales preservan la información relativa a la localización de cada parche en la imagen original.

Posteriormente, esta secuencia es procesada por múltiples capas de autoatención, donde cada parche puede intercambiar información con todos los demás. Este mecanismo permite construir representaciones con contexto global, capturando relaciones de largo alcance.

IV-A2. Decoder convolucional y reconstrucción espacial

La salida del encoder, expresada como una secuencia de vectores latentes, es reestructurada nuevamente en forma de mapa bidimensional de características. A partir de esta representación de baja resolución espacial, el decoder convolucional realiza una reconstrucción progresiva hasta alcanzar la resolución original de la imagen.

Este proceso se lleva a cabo mediante una serie de etapas de refinamiento y reescalado, en las cuales se combinan convoluciones para la extracción local de características con operaciones de interpolación para el aumento gradual de la resolución. En la etapa final, una proyección lineal por píxel permite obtener, para cada clase, un mapa de activación que representa la pertenencia de cada píxel a las distintas regiones de interés.

IV-A3. Integración en el pipeline de AutoML

La arquitectura ViT descrita se integra en el pipeline de AutoML mediante un componente que encapsula tanto el proceso de entrenamiento como la generación de predicciones. Este componente actúa como intermediario entre el modelo de segmentación y el resto del framework, garantizando que el ViT pueda ser tratado de manera homogénea junto con otras arquitecturas.

Durante el entrenamiento, se emplea un esquema supervisado estándar para segmentación multiclase, utilizando funciones de pérdida adecuadas para este tipo de problema y validación sobre datos no vistos para monitorear el desempeño. En la fase de inferencia, las salidas continuas del modelo son convertidas en máscaras discretas mediante una asignación por máxima activación, produciendo mapas de segmentación directamente comparables con las máscaras de referencia.

Esta estrategia combina la capacidad de los Vision Transformers para modelar contexto global con la precisión espacial de decodificadores convolucionales, resultando adecuada para problemas de segmentación.

IV-B. Swin Transformer

La estrategia de segmentación basada en Swin Transformers se apoya en una arquitectura jerárquica de atención local con ventanas desplazadas, adaptada para producir mapas de

segmentación densos a partir de imágenes de microscopía electrónica.

El modelo recibe como entrada imágenes bidimensionales de tamaño fijo, representadas como tensores en escala de grises. En casos donde la información de entrada presenta múltiples canales, se aplica una conversión previa para garantizar consistencia con la arquitectura. La salida del modelo es un mapa de segmentación multiclase con la misma resolución espacial que la imagen de entrada, en el que cada píxel es asignado a una de las clases consideradas.

El procesamiento se inicia mediante una etapa de patch embedding jerárquica, en la cual la imagen se descompone en parches de pequeño tamaño que son proyectados a un espacio latente inicial. A diferencia de los Vision Transformers clásicos, el modelo Swin organiza estas representaciones en una estructura piramidal de múltiples etapas, donde la resolución espacial se reduce progresivamente y la dimensión de las características aumenta. En cada etapa, la atención se calcula de forma local dentro de ventanas, alternando ventanas fijas y desplazadas, lo que permite capturar dependencias espaciales tanto locales como de mayor alcance con un costo computacional controlado.

Las características extraídas por el backbone Swin corresponden a una representación de baja resolución espacial pero alta riqueza semántica. Para su uso en segmentación densa, estas representaciones se reorganizan y se adaptan mediante una etapa intermedia de refinamiento y reescalado, con el objetivo de armonizar la resolución con la del decodificador.

Posteriormente, un decodificador convolucional progresivo reconstruye la resolución original de la imagen. Este decodificador realiza una serie de operaciones de convolución, normalización y reescalado espacial, transformando las representaciones latentes jerárquicas en predicciones a nivel de píxel. La última proyección produce un conjunto de mapas de probabilidad, uno por clase, que se convierten en máscaras discretas mediante una asignación por máxima probabilidad.

El modelo se entrena de extremo a extremo utilizando una función de pérdida de entropía cruzada multiclase, optimizando directamente la coherencia entre las máscaras predichas y las anotaciones de referencia. Durante la evaluación, el modelo genera pares de máscaras predichas y reales, lo que permite analizar su desempeño tanto de forma cuantitativa como cualitativa.

Esta estrategia aprovecha la naturaleza jerárquica y la atención localizada del Swin Transformer para modelar de manera eficiente estructuras complejas a múltiples escalas, resultando especialmente adecuada para tareas de segmentación en imágenes con alta variabilidad textural y patrones multiescala.

IV-C. Quadtree

La estrategia de segmentación basada en quadtree adopta un enfoque jerárquico y recursivo, en el cual la imagen se descompone progresivamente en regiones cuadradas de menor tamaño en función de la confianza de un clasificador subyacente. Este método no pertenece al estado del arte en segmentación, pero permite reutilizar modelos de clasificación preentrenados para abordar tareas de segmentación bajo restricciones computacionales.

El modelo opera sobre imágenes de tamaño fijo y genera máscaras de segmentación a nivel de píxel mediante la clasificación iterativa de regiones. Cada región es evaluada por un clasificador que asigna una etiqueta y una confianza asociada. Si la confianza supera un umbral predefinido, la región se considera homogénea y se asigna directamente a la máscara de salida. En caso contrario, la región se subdivide en cuatro cuadrantes, repitiendo el proceso de forma recursiva.

El criterio de parada de la recursión se define por tres condiciones: alcanzar un nivel mínimo de confianza, llegar a un tamaño mínimo de región o exceder una profundidad máxima de subdivisión. De esta forma, el modelo equilibra precisión espacial y costo computacional, evitando una subdivisión excesiva en regiones con información limitada.

El proceso de entrenamiento se divide conceptualmente en dos etapas. En primer lugar, el clasificador utilizado por el esquema quadtree es entrenado de manera independiente empleando un conjunto de datos de clasificación externo. Este clasificador puede estar basado en redes convolucionales, arquitecturas transformer o cualquier otro modelo, lo que permite transferir conocimiento desde tareas de clasificación hacia la segmentación.

En segundo lugar, el modelo de segmentación ajusta sus hiperparámetros estructurales, en particular el umbral de confianza, el tamaño mínimo de región y la profundidad máxima del quadtree. Cuando se habilita el modo AutoML, estos hiperparámetros son optimizados mediante una metaheurística de recocido simulado (simulated annealing), utilizando un subconjunto de validación derivado del dataset de segmentación. La optimización se guía por una métrica seleccionada previamente, permitiendo adaptar el comportamiento del quadtree al compromiso deseado entre detalle espacial y generalización.

Una vez finalizado el proceso de optimización, el modelo se configura con la mejor combinación de hiperparámetros encontrada y se evalúa sobre el conjunto completo de entrenamiento.

V. MODELOS DE CLASIFICACIÓN PARA SEGMENTACIÓN

En esta sección se describen los modelos de clasificación empleados como componentes fundamentales de las estrategias de segmentación guiada basadas en particionado espacial. A diferencia de los enfoques de segmentación directa, estos métodos delegan la toma de decisiones semánticas a clasificadores entrenados para discriminar entre zonas frágiles y dúctiles, cuya salida se utiliza posteriormente para construir máscaras de segmentación a distintas escalas.

El objetivo de esta sección es caracterizar los clasificadores utilizados, independientemente del esquema de particionado específico en el que se integran. De este modo, se separa explícitamente el análisis del modelo de clasificación del mecanismo geométrico de segmentación, permitiendo evaluar de forma aislada el impacto de la arquitectura del clasificador en el desempeño final del sistema. Las subsecciones siguientes presentan los distintos modelos considerados.

V-A. Clasificador CNN

El clasificador basado en Redes Neuronales Convolucionales (CNN) fue diseñado para clasificar regiones de tamaño

variable extraídas por el algoritmo Quadtree.

La arquitectura sigue un patrón de extracción de características seguido de clasificación:

$$\begin{aligned} \text{Entrada} &\rightarrow \text{Bloques Conv.} \rightarrow \text{Pooling Global} \\ &\rightarrow \text{Clasificador} \rightarrow \text{Softmax} \end{aligned} \quad (2)$$

Cada bloque convolucional aplica la siguiente secuencia:

1. Convolución 3×3 con padding=1
2. Batch Normalization
3. Activación ReLU
4. Convolución 3×3 con padding=1
5. Batch Normalization
6. Activación ReLU
7. Max Pooling 2×2 con stride=2

Se usa un kernel de 3×3 porque es el tamaño mínimo que captura información espacial (arriba, abajo, izquierda, derecha y diagonales). Al apilar múltiples capas con kernels pequeños se obtiene un campo receptivo grande con menos parámetros que usando un kernel grande directamente.

El padding de 1 píxel mantiene las dimensiones espaciales constantes después de cada convolución, lo que simplifica el diseño de la red.

Batch Normalization se incluye para acelerar el entrenamiento y estabilizar los gradientes, permitiendo usar tasas de aprendizaje más altas.

Se usan dos convoluciones por bloque antes del pooling para aumentar la capacidad de la red sin reducir las dimensiones espaciales prematuramente.

El parámetro `num_blocks` controla la profundidad de la red. Como cada bloque reduce las dimensiones a la mitad mediante Max Pooling, el tamaño mínimo de entrada es $2^n \times 2^n$ píxeles, donde n es el número de bloques.

Cuadro I
RELACIÓN ENTRE NÚMERO DE BLOQUES Y TAMAÑO MÍNIMO DE ENTRADA

num_blocks	Tamaño mínimo	Canales finales
2	4×4	64
3	8×8	128
4	16×16	256
5	32×32	512

El valor por defecto es 3 bloques, lo que permite clasificar regiones de 8×8 píxeles. Este tamaño fue elegido porque el Quadtree puede subdividir hasta regiones pequeñas, y 8×8 se consideró suficiente para contener información visual distinguible mientras permite segmentación detallada.

El número de filtros se duplica en cada bloque, comenzando desde 32:

$$\text{filtros en bloque } i = 32 \times 2^{i-1} \quad (3)$$

Esta progresión compensa la reducción de dimensiones espaciales: a medida que la imagen se hace más pequeña, se aumentan los canales para mantener la capacidad de representación.

Antes de la clasificación se aplica `AdaptiveAvgPool2d((1,1))`, que reduce cualquier tamaño espacial a 1×1 calculando el promedio de cada canal.

Esta capa permite que la red acepte entradas de cualquier tamaño (siempre que cumplan el mínimo). Sin importar si la entrada es 8×8 o 256×256 , la salida siempre es un vector de tamaño fijo que puede procesarse por las capas densas.

La cabeza de clasificación consiste en:

1. Aplanamiento del tensor
2. Dropout (50 %)
3. Capa densa con reducción a 1/4 de los canales
4. Activación ReLU
5. Dropout (25 %)
6. Capa densa final a 3 clases

El Dropout previene sobreajuste, que es una preocupación con datasets pequeños de imágenes SEM. La capa intermedia reduce la dimensionalidad gradualmente antes de la clasificación final.

La red produce probabilidades mediante Softmax, donde la probabilidad más alta indica la clase predicha y su valor representa la confianza. Esta confianza es usada por el Quadtree para decidir si subdividir la región o aceptar la clasificación.

El entrenamiento usa Cross-Entropy Loss y el optimizador Adam. Las imágenes se agrupan por tamaño para formar batches, ya que las regiones del Quadtree tienen dimensiones variables y solo se pueden apilar en un tensor imágenes del mismo tamaño.

V-A1. Clasificador ViT

Se empleó un clasificador basado en Vision Transformer (ViT) como mecanismo de inferencia semántica a nivel de región, destinado a transferir conocimiento desde una tarea de clasificación supervisada hacia esquemas de segmentación guiada. Este modelo no actúa como segmentador directo, sino como un estimador de clase aplicado sobre regiones espaciales extraídas de la imagen, cuyo resultado se utiliza posteriormente para construir máscaras de segmentación mediante estrategias geométricas.

El clasificador adopta una arquitectura ViT estándar con token de clasificación (*CLS*), en la que la imagen de entrada se divide en parches no solapados y se proyecta a un espacio embebido mediante una capa convolucional equivalente a la etapa de *patch embedding*. La secuencia resultante, enriquecida con embeddings posicionales, es procesada por un codificador Transformer profundo, y la representación final asociada al token *CLS* se utiliza como descriptor global de la región analizada. Esta representación se proyecta a un espacio de clases mediante una cabeza de clasificación multicapa.

Desde el punto de vista operativo, el modelo se integra como un clasificador de regiones de tamaño fijo, lo que requiere un preprocesamiento consistente que normaliza, ajusta el número de canales y reescala cada región al tamaño de entrada esperado por la arquitectura. Esta restricción permite garantizar una semántica homogénea en la inferencia, independientemente del tamaño real de la región en la imagen original.

VI. PIPELINE DE AUMENTACIÓN DE DATOS

Se implementó un conjunto completo de técnicas de aumentación de datos con el objetivo de incrementar la diversidad del dataset y mejorar la generalización de los modelos. La

estrategia combinó transformaciones geométricas, fotométricas y específicas para SEM, aplicadas de manera secuencial y probabilística según un pipeline de aumentación.

VI-1. Aumentaciones Geométricas

Afectan tanto a imágenes como a máscaras, manteniendo la alineación espacial:

- Rotación: Rota la imagen y la máscara dentro de un rango definido, simulando diferentes orientaciones de la muestra.
- Volteo: Invierte horizontal o verticalmente.
- Escalado: Realiza zoom in/out manteniendo el tamaño original.
- Traslación: Desplaza la imagen y máscara horizontal y/o verticalmente.
- Recorte aleatorio: Extrae subregiones aleatorias, redimensionadas al tamaño original.

Estas transformaciones permiten al modelo generalizar a variaciones espaciales de la muestra.

VI-2. Aumentaciones Fotométricas

Modifican únicamente las imágenes, simulando variaciones en iluminación y ruido, sin afectar las máscaras:

- Brillo.
- Contraste.
- Corrección gamma.
- Ruido gaussiano.
- Desenfoque gaussiano.

Estas técnicas permiten al modelo aprender invariancia frente a condiciones de adquisición o ruido instrumental.

VI-3. Aumentaciones Específicas para SEM

Diseñadas para capturar artefactos y características típicas de imágenes SEM:

- Deformación elástica (ElasticDeformationAugmentor): Simula variaciones naturales en texturas de roca o deformaciones durante la adquisición.
- Ecuilización adaptativa de histograma (AdaptiveHistogramEqualizationAugmentor): Mejora contraste local mediante CLAHE.
- Artefactos de carga: Añade manchas típicas de muestras no conductivas.
- Ruido de líneas de escaneo (ScanLineNoiseAugmentor): Simula líneas de escaneo o barrido que aparecen en SEM.

Estas técnicas aumentan la robustez del modelo frente a artefactos específicos de la microscopía electrónica.

VI-4. Composición de Aumentaciones

Se implementaron métodos compuestos que permiten combinar varias aumentaciones para crear un pipeline complejo y controlado:

- Secuencial: Aplica múltiples aumentaciones de manera secuencial.
- Aplicación probabilística: Aplica una aumentación con una probabilidad determinada, introduciendo variabilidad controlada.
- Selección aleatoria: Elige una aumentación de un conjunto con probabilidades ponderadas.

Estas estrategias permiten generar un dataset enriquecido sin comprometer la coherencia entre imágenes y máscaras.

VI-A. Métricas de Evaluación

La evaluación del desempeño de los modelos de segmentación se diseñó de forma modular y extensible, permitiendo calcular múltiples métricas a partir de una representación común de las predicciones y las máscaras de referencia. Todo el proceso es coordinado por un nodo evaluador central (*EvaluatorNode*), cuyo objetivo es desacoplar la generación de predicciones del cálculo de métricas, facilitando comparaciones consistentes entre modelos y configuraciones experimentales.

Para cada imagen del conjunto de validación, el modelo de segmentación genera una máscara predicha, la cual se empareja con su máscara real correspondiente. Este proceso produce una colección de pares $(\hat{M}, M)$, denominada *MaskPairs*. En escenarios con validación cruzada k-fold, estas colecciones se agrupan por pliegue, resultando en una estructura del tipo lista de listas; en el caso simple de train/validation split, existe un único conjunto de pares.

Esta estructura es la entrada principal del *EvaluatorNode*, que no calcula métricas directamente, sino que delega el cómputo a evaluadores especializados. Cada evaluador implementa una métrica específica y devuelve un único valor escalar, el cual es finalmente agregado en un diccionario de resultados que resume el desempeño del modelo.

La mayoría de las métricas utilizadas se fundamentan en interpretar la segmentación como un problema de clasificación a nivel de píxel. Para una clase dada, cada píxel de la máscara predicha se compara con la máscara real y se clasifica como verdadero positivo (TP), falso positivo (FP), falso negativo (FN) o verdadero negativo (TN). Los evaluadores acumulan estos conteos a lo largo de todas las imágenes antes de calcular la métrica final.

VI-A1. Intersección sobre Unión (IoU) y coeficiente Dice

El *Intersection over Union* (IoU) y el coeficiente *Dice* son las métricas principales para cuantificar el solapamiento entre predicción y referencia. El IoU se define como la razón entre la intersección y la unión de ambas máscaras, penalizando tanto falsas detecciones como omisiones. El Dice, estrechamente relacionado, pondera el solapamiento relativo y puede interpretarse como una forma del F1-score a nivel de píxel, siendo ligeramente más sensible a errores de clasificación.

Estas métricas se calculan de tres maneras complementarias: (i) por clase, evaluando individualmente cada etiqueta; (ii) promedio macro, donde se promedia el valor de cada clase sin ponderación, ofreciendo una visión balanceada incluso ante desbalances severos; (iii) promedio ponderado, donde cada clase contribuye según su frecuencia en el conjunto de datos, reflejando el desempeño global pero siendo sensible a clases dominantes como el fondo.

VI-A2. Precisión y Recall

La precisión y el recall permiten un diagnóstico más fino de los errores del modelo. La precisión mide la confiabilidad de las predicciones positivas, mientras que el recall cuantifica la capacidad del modelo para detectar todos los píxeles relevantes de una clase. Al igual que en IoU y Dice, se reportan promedios macro para analizar el comportamiento medio del modelo en todas las clases, independientemente de su tamaño relativo.

VI-A3. Cohesión de la máscara

Además de las métricas clásicas basadas en coincidencia píxel a píxel, se incorpora una métrica estructural denominada *Mask Cohesion*. Esta se implementa mediante un autoencoder convolucional entrenado exclusivamente con máscaras reales del conjunto de datos. El autoencoder aprende una representación compacta de las estructuras válidas presentes en las segmentaciones de referencia.

Durante la evaluación, las máscaras predichas se reconstruyen a través de este autoencoder y se calcula el error de reconstrucción. Un error bajo indica que la predicción presenta estructuras coherentes y plausibles, similares a las observadas en las máscaras reales, mientras que un error alto sugiere ruido, fragmentación o patrones no realistas. Esta métrica complementa a IoU y Dice al capturar aspectos cualitativos de la segmentación que no siempre se reflejan en métricas puramente basadas en solapamiento.

Este conjunto de métricas proporciona una evaluación integral del desempeño, abarcando exactitud cuantitativa, balance entre clases y calidad estructural de las segmentaciones producidas.

VII. RESULTADOS

VIII. CONCLUSIONES